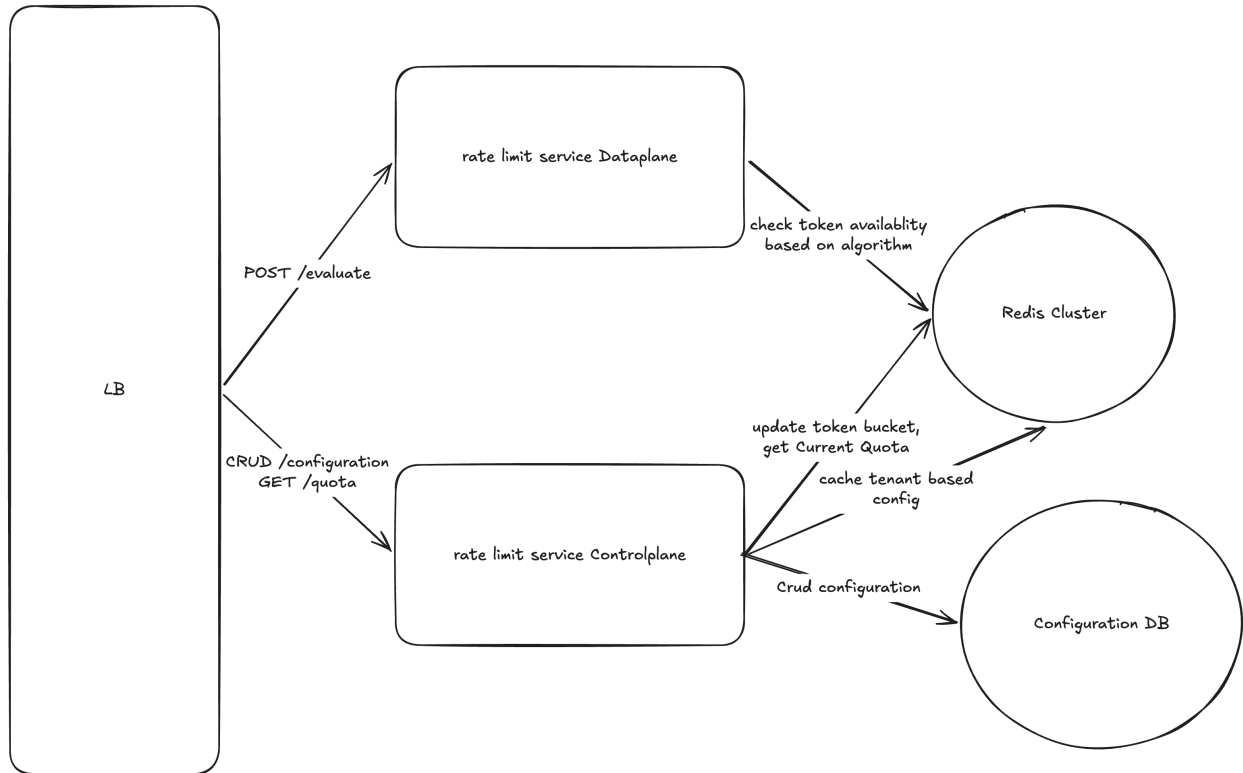


Digital Ocean - Distributed Rate Limiter

Specifications/Expectations

1. Quota enforcement: Accept an identifier (tenant id) and the resource namespace. return an allow deny decision with remaining quota and reset time, support at least two algorithm choices (token bucket and sliding window)
2. Configuration api: Allow limit rules to be created, updated and deleted at runtime without restarting the service. Rules should be for burst and sustained rate distinctions
3. Multi instance correctness : the service should provide correct decisions even when two or more instances share state, not just when running a single process
4. Observable quota state - provide an endpoint returning current quota consumption, remaining capacity and reset timestamps for given identifier

High Level Design



APIs/Models

API:

1. **POST /evaluate**
 evaluationRequestDTO {
 namespace: string,
 tenantId: uuid
 }
2. **GET /configuration/{namespace}/{tenantId}**
 configurationDTO {
 rateLimitConfig: RateLimitConfig,
 tokenConfiguration: TokenConfiguration,
 tenantId: UUID,
 namespace: String,
 }
3. **PUT /configuration/{namespace}/{tenantId}**

4. POST /configuration/{namespace}/{tenantId}
5. DELETE /configuration/{namespace}/{tenantId}
6. Get /quota/{namespace}/{tenantId}
quotaDTO {
 burstableTokenAvailability: TokenAvailability
 sustainedTokenAvailability: TokenAvailability
 namespace: string
 tenantId: UUID
}

Models

1. evaluationRequest {
 namespace: string,
 tenantId: uuid
}
2. Configuration {
 burstableRateLimitConfig: RateLimitConfig,
 fixedRateLimitConfig: RateLimitConfig,
 tokenCount: long,
 tenantId: UUID, <combined primary>
 namespace: String, <combined secondary>
}
3. RateLimitAlgorithm {
 TOKEN_BUCKET,
 FIXED_WINDOW
}
4. RateLimitConfig {
 rateLimitAlgorithm: RateLimitAlgorithm
 availableToken: long
 timeWindowInSeconds: long
}
5. Quota {
 tenantId: UUID <combined primary>

```

namespace: String, <combined primary>
availableToken: long,
currentConsumedToken: long,
resetTimestamp: Date
}

```

```

6. tokenAvailability {
    availableToken: long
    remainingToken: long
    resetTimestamp: Date
}

```

Low Level Design

Rate Limiter

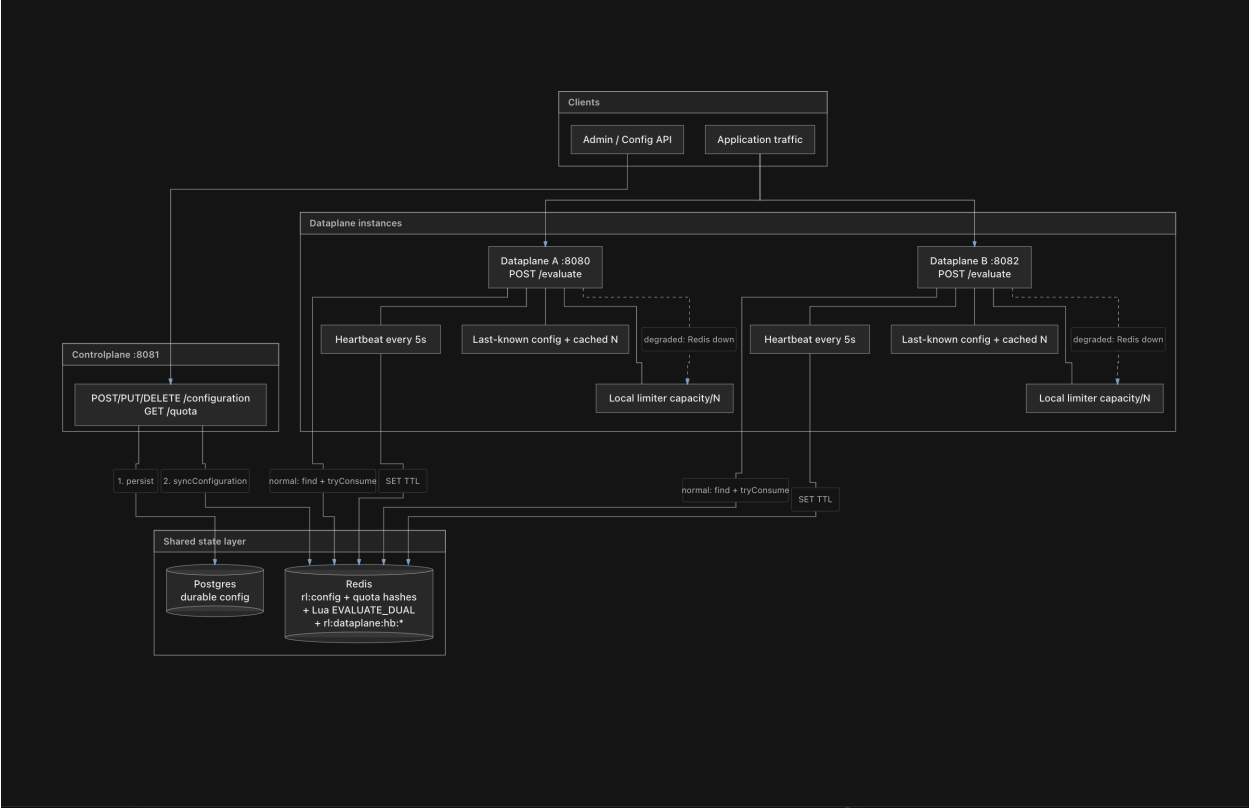
```

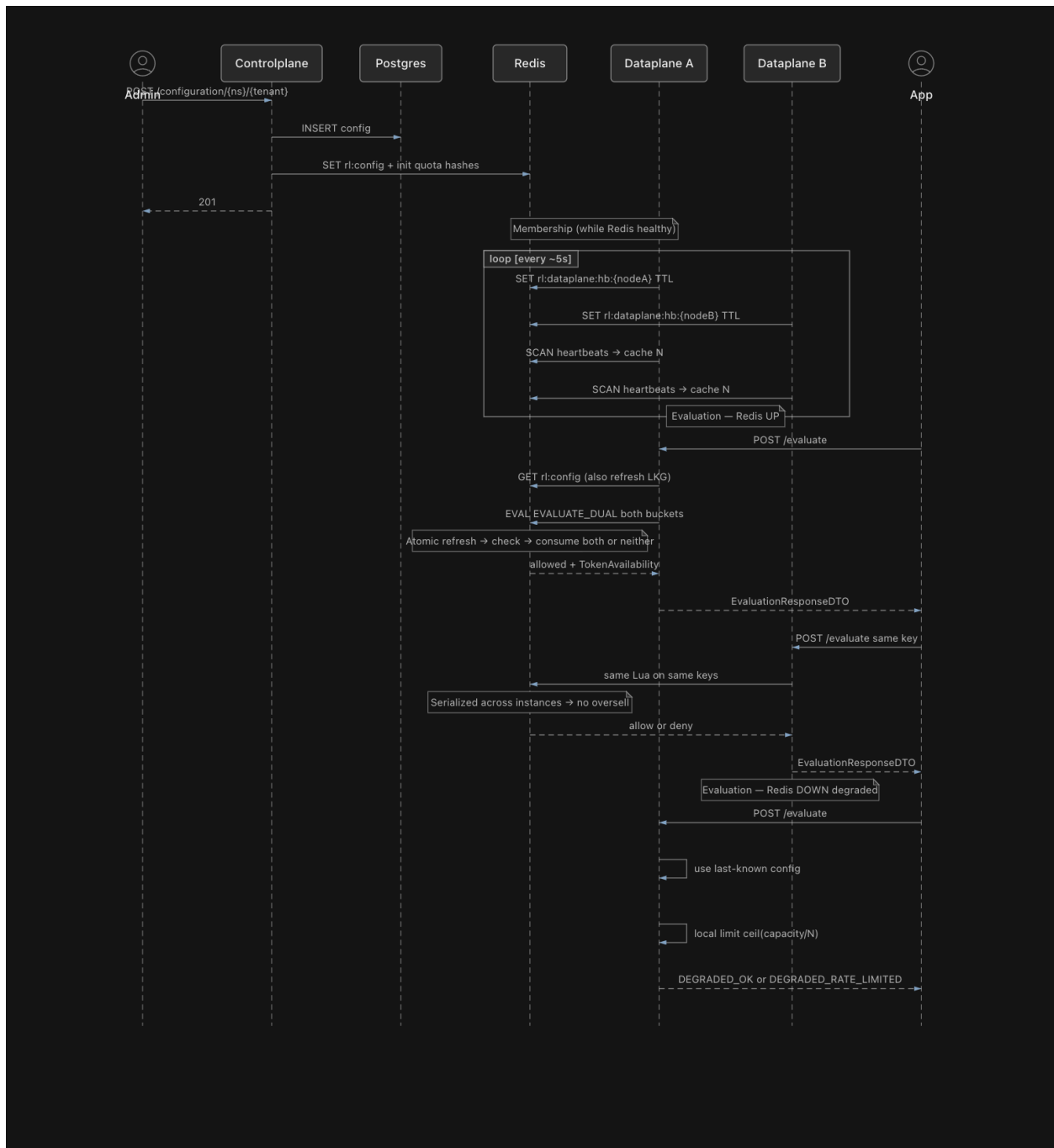
-> rate-limiter-common
    -> DTO
        -> ConfigurationDTO
        -> EvaluationRequestDTO
        -> QuotaDTO
    -> Enums
    -> Model
    -> Redis
        -> RateLimitScripts
-> rate-limiter-dataplane
    -> Algorithm
        -> AlgorithmEvaluatorFactory
            | -> AlgorithmEvaluator -> FixedWindowEvaluator,
SlidingWindowEvaluator, TokenBucketEvaluator
    -> Config
        -> InMemory, Redis
    -> Controller
        -> EvaluatorController
    -> Service
        -> evaluatorService

```

```
-> rateLimitEvaluator
-> store
    -> Configurationstore -> InMemory,Redis
    -> DualBucketQuotaStore -> InMemory,Redis
-> rate-limiter-controlplane
    -> Config
    -> ConfigurationController
    -> Entity
    -> Redis
        -> RedisConfigurationStore
    -> Repository
        -> ConfigurationRepository
    -> Service
        -> ConfigurationService
```

Request Flow





Design Considerations

Scalability

We have separated out services into controlplane and dataplane, and can be scaled up independently.

Limitations

Scaling up of dataplane highly depends on the number of connections it can hold with redis

Consistency

Dataplane

Dataplane contacts Redis, to fetch tokens based on algorithm. Since Redis is single threaded, the execution of the lua scripts that consumes and populates a token are gurateed to be atomic.

Controlplane

Controlplane contacts both Postgressql and redis, sql to populate the configurations, so we have a consistent information store, and redis, to cater to high demand workflows. Configurations flow from Controlplane to the dataplane, and it is done in a single transaction.

Limitations

Redis serves as a source of truth for all operations, thereby tightly coupling our service with redis

Availability

Design is focused on high availablity of dataplane. But it is still tied up to redis.

Disaster Recovery

Since, we have tight coupling with redis, we need to have a disaster recovery when redis goes down.

1. Heartbeat Daemon finds the number of active dataplane instances interacting with redis, and makes a local cache
2. It also has a config for tenant/namespace available locally

3. When Redis goes down, it will fall back to in memory store, where it uses configurationsSet/(last known dataplane nodes) as an upper limit, and serves traffic with limited availability until redis comes back up